BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG

PH TRƯỜNG ĐẠI HỌC THỦY LỢI

BM CÔNG NGHỆ THÔNG TIN



BÁO CÁO MÔN CHUYÊN ĐỀ CÔNG NGHỆ THÔNG TIN

Tên đề tài:

XÂY DỰNG ỨNG DỤNG HỖ TRỢ DỊCH VÀ HỌC NGÔN NGỮ KÝ HIỆU DÀNH CHO NGƯỜI KHIẾM THÍNH VÀ NGƯỜI CÂM

Giảng viên hướng dẫn	*T.s Hoàng Văn Quý*
Sinh viên thực hiện	*Nguyễn Ngọc Châu Giang*
Mssv	*2351067092*
Lớp	*S26-65CNTT*

2026

MỤC LỤC

Mở đầu 5
1. Lý do chọn đề tài 5
2. Bài toán 5
3. Dữ liệu sử dụng 5
4. Kết quả chính 6
5. Sản phẩm dự kiến 6
Chương 1. Cơ sở lý thuyết 7
1.1. Ngôn ngữ Ký hiệu Việt Nam 7
1.2. YOLO26 và ước lượng tư thế 7
1.3. MediaPipe Hands và Face Landmarker 7
1.4. Chuẩn hóa tọa độ keypoint 8
1.5. Mạng hồi quy LSTM và GRU 8
1.6. Attention pooling 8
1.7. Tăng cường dữ liệu và chuẩn hóa tốc độ 9
1.8. Hàm mất mát và tối ưu 9
1.9. Tổng hợp tiếng nói (TTS) 9
1.10. Độ đo đánh giá 9
Chương 2. Đề xuất hệ thống 10
2.1. Kiến trúc tổng thể 10
2.2. Các module 10
2.3. Vector đặc trưng mỗi frame 12
2.4. Mô hình nhận diện 12
2.5. Giải mã liên tục (webcam và video dài) 13
2.6. Dịch sang tiếng Việt 13

2.7. Đọc thành tiếng 13
Chương 3. Kết quả và thực nghiệm 14
3.1. Môi trường thực nghiệm 14
3.2. Dữ liệu và cách chia 14
3.3. Kiểm thử pipeline 15
3.4. Kết quả nhận diện trên tập test 15
3.5. Hiệu quả của ngưỡng tin cậy 15
3.6. Tốc độ xử lý 16
3.7. Biểu đồ kết quả 16
3.8. Thảo luận và hạn chế 16
3.9. Thực nghiệm đang thực hiện 17
Kết luận 18
1. Kết quả đạt được 18
2. Hướng phát triển 18
Tài liệu tham khảo 20

LỜI CẢM ƠN

Trong khoảng thời gian làm đồ án, em đã nhận được nhiều sự giúp đỡ, đóng góp ý kiến và sự dẫn dắt chỉ bảo nhiệt tình của thầy. Em xin gửi lời cảm ơn chân thành đến giáo viên hướng dẫn - T.s Hoàng Văn Quý - Giảng viên môn CHUYÊN ĐỀ CNTT - Phân Hiệu Trường Đại học Thủy Lợi người đã tận tình hướng dẫn, chỉ bảo em trong suốt quá trình làm khoá luận.

Với điều kiện về thời gian cũng như lượng kiến thức về đề tài rất rộng mà kinh nghiệm còn hạn chế của một học viên, đồ án này không thể tránh được những thiếu sót. Em rất mong nhận được sự chỉ bảo, đóng góp ý kiến của thầy để em có điều kiện bổ sung, nâng cao kiến thức của mình, phục vụ tốt hơn công tác thực tế sau này.

Mở đầu

Đề tài xây dựng một hệ thống nhận video người ký Ngôn ngữ Ký hiệu Việt Nam (VSL), nhận diện ký hiệu, dịch thành câu tiếng Việt và đọc thành giọng nói. Mô hình đạt độ chính xác 95,4% và F1-macro 0,939 trên 568 video kiểm thử thuộc 82 từ. Hệ thống chạy khoảng 9 FPS trên CPU và được đóng gói thành ứng dụng Android.

1. Lý do chọn đề tài

Người Điếc ở Việt Nam giao tiếp chủ yếu bằng VSL, trong khi phần lớn người nghe không hiểu ngôn ngữ này. Thông dịch viên ký hiệu rất ít, nên các tình huống hằng ngày như khám bệnh, đi học hay làm thủ tục thường gặp rào cản. Một công cụ dịch tự động, dù chỉ ở mức từ vựng, đã có thể hỗ trợ các cuộc giao tiếp ngắn.

2. Bài toán

- **Đầu vào:** video (MP4/AVI/MOV) hoặc luồng webcam/camera điện thoại có một người thực hiện ký hiệu.
- **Đầu ra:** chuỗi nhãn ký hiệu kèm độ tin cậy, câu tiếng Việt có dấu câu và âm thanh tiếng Việt.
- **Phạm vi:** nhận diện ký hiệu mức từ (isolated sign recognition). Câu nhiều từ được ghép bằng cửa sổ trượt. Hệ thống chưa phải dịch máy ngôn ngữ ký hiệu liên tục ở mức câu.
- **Ràng buộc:** dùng YOLO26 làm mô hình thị giác chủ đạo, có mô hình chuỗi thời gian (LSTM/GRU), chạy được thời gian thực, ưu tiên Windows.

3. Dữ liệu sử dụng

Bảng 1. Thống kê bộ dữ liệu sử dụng trong bài toán

Bộ dữ liệu	Quy mô	Vai trò trong đề tài
ViSignLanguage-Video	khoảng 3.875 clip MP4, 100 từ, 224×224, CC BY 4.0	Dữ liệu chính, chia train/val/test 70/15/15 theo từng nhãn
QIPEDC (Bộ GD&ĐT, bản crawl trên Kaggle)	4.362 video, khoảng 1–3 video/từ, có biến thể Bắc/Trung/Nam	Bổ sung vào tập train cho các từ trùng với ViSign
Dữ liệu mẫu tổng hợp	100 video sinh bằng biến đổi hình học	Chỉ dùng kiểm thử pipeline, không phải ký hiệu thật

Sau khi lọc clip lỗi và bỏ các nhãn có dưới 8 clip, tập dữ liệu còn 82 từ.

4. Kết quả chính

- Pipeline hoàn chỉnh: đọc video, YOLO26-pose, MediaPipe Hands/Face, vector 291 chiều mỗi frame, BiGRU có attention, dịch tiếng Việt, TTS.
- Trên tập test: accuracy 95,42%, F1-macro 0,939. Với ngưỡng tin cậy 0,70, hệ thống trả lời 91,0% số video và đúng 99,6% trong số đó.
- Nhận diện thời gian thực qua webcam với cơ chế xác nhận nhiều cửa sổ và tự kết thúc câu. Có giao diện desktop PyQt5.

5. Sản phẩm dự kiến

Sản phẩm cuối là ứng dụng Android **"Dịch Ký hiệu"**. Người dùng đăng nhập, quay video 1–3 giây, rồi nhận caption tiếng Việt, độ tin cậy và giọng đọc. Lịch sử dịch được lưu theo tài khoản, và người dùng có thể báo "sai rồi" kèm nhãn đúng để cải thiện dữ liệu. Giai đoạn tiếp theo mở rộng từ vựng lên khoảng 450 từ với bộ VieSign-SI447, đánh giá trên người ký chưa từng thấy.

Chương 1. Cơ sở lý thuyết

Nhận diện ký hiệu là bài toán phân loại chuỗi theo thời gian. Một mô hình thị giác trích ra vị trí thân và tay ở từng frame, sau đó một mô hình chuỗi học ý nghĩa từ chuyển động của các vị trí đó.

1.1. Ngôn ngữ Ký hiệu Việt Nam

VSL là ngôn ngữ thị giác có ngữ pháp riêng, không phải tiếng Việt được diễn đạt bằng tay. Mỗi ký hiệu kết hợp hình dạng bàn tay, vị trí so với cơ thể, hướng lòng bàn tay, chuyển động và biểu cảm khuôn mặt. Hai ký hiệu có thể cùng hình dạng tay và chỉ khác hướng chuyển động, nên một ảnh tĩnh không đủ để phân biệt. VSL còn có khác biệt vùng miền: các phương ngữ Hà Nội, Hải Phòng, TP.HCM có nhiều ký hiệu khác nhau cho cùng một từ. Trong VSL, từ để hỏi và phủ định thường đứng cuối câu.

1.2. YOLO26 và ước lượng tư thế

YOLO (You Only Look Once) là họ mô hình phát hiện đối tượng một giai đoạn: một lượt lan truyền cho ra cả khung và lớp của đối tượng. YOLO26 của Ultralytics là phiên bản mới nhất, hỗ trợ cả phát hiện, phân vùng, phân loại, ước lượng tư thế và khung xoay trong một kiến trúc. Các cải tiến chính (Ultralytics, arXiv 2510.09653):

- Suy luận **end-to-end không cần NMS** (Non-Maximum Suppression), giảm độ trễ khi triển khai.
- Bỏ Distribution Focal Loss (DFL), đơn giản hóa việc xuất sang ONNX, TFLite.
- ProgLoss và STAL giúp huấn luyện ổn định và nhận đối tượng nhỏ tốt hơn. Bộ tối ưu MuSGD giúp hội tụ nhanh hơn.

Mô hình **YOLO26-pose** (`yolo26n-pose.pt`) dự đoán khung người và **17 khớp theo chuẩn COCO**: mũi, mắt, tai, vai, khuỷu, cổ tay, hông, gối, cổ chân. Mô hình này **không có điểm ngón tay**. YOLO26 xử lý từng ảnh độc lập, nên không tự hiểu được ý nghĩa của một chuỗi ký hiệu. Vì vậy, đề tài dùng YOLO26 làm bộ phát hiện không gian và để mô hình chuỗi ở phía sau xử lý thời gian.

1.3. MediaPipe Hands và Face Landmarker

MediaPipe Hand Landmarker định vị **21 điểm 3D trên mỗi bàn tay** (cổ tay và 4 khớp trên mỗi ngón) qua hai bước: phát hiện lòng bàn tay, rồi hồi quy các điểm mốc trên vùng cắt. Face Landmarker cho 478 điểm khuôn mặt; đề tài chọn 20 điểm ở lông mày, mắt, mũi, môi và cằm để biểu diễn biểu cảm. Từ bản 0.10.30, MediaPipe bỏ API `mp.solutions` cũ, nên hệ thống dùng Tasks API (`HandLandmarker`, `FaceLandmarker`) ở chế độ VIDEO.

1.4. Chuẩn hóa tọa độ keypoint

Tọa độ pixel phụ thuộc vị trí đứng, khoảng cách tới camera và độ phân giải. Ta chuẩn hóa mọi điểm p theo hệ trục gắn với cơ thể: gốc c là trung điểm hai vai, đơn vị s là khoảng cách hai vai.

$$\hat{p} = \frac{p - c}{s}, \qquad c = \frac{p_{LS} + p_{RS}}{2}, \qquad s = \| p_{LS} - p_{RS} \|_2$$

Hình dạng bàn tay được biểu diễn thêm bằng tọa độ cục bộ so với cổ tay q_0, chia cho độ dài từ cổ tay tới khớp gốc ngón giữa q_9. Nhờ vậy, cùng một hình dạng tay cho cùng một vector, dù tay đặt ở đâu.

$$\tilde{q}_k = \frac{q_k - q_0}{\|q_9 - q_0\|_2}, \qquad k = 0, \ldots, 20$$

1.5. Mạng hồi quy LSTM và GRU

Mạng hồi quy (RNN) xử lý chuỗi $x_1, \ldots, x_T$ bằng trạng thái ẩn h_t cập nhật qua từng bước. RNN thuần bị tiêu biến gradient trên chuỗi dài. **LSTM** (Hochreiter & Schmidhuber, 1997) khắc phục bằng ô nhớ c_t và ba cổng quên, vào, ra:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \quad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i), \quad o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tanh(W_c x_t + U_c h_{t-1} + b_c), \qquad h_t = o_t \odot \tanh(c_t)$$

GRU (Cho và cộng sự, 2014) gộp thành hai cổng cập nhật z_t và đặt lại r_t. GRU ít tham số hơn LSTM khoảng 25% và thường cho kết quả tương đương trên tập dữ liệu nhỏ:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z), \qquad r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$$
$$\tilde{h}_t = \tanh(W_h x_t + U_h (r_t \odot h_{t-1}) + b_h), \qquad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Bản **hai chiều** (bidirectional) chạy một lần theo chiều xuôi và một lần theo chiều ngược, rồi nối hai trạng thái. Mỗi bước vì thế biết cả chuyển động trước và sau nó.

1.6. Attention pooling

Không phải frame nào cũng mang thông tin như nhau: đoạn tay đưa lên và hạ xuống ít ý nghĩa hơn đoạn giữa. Attention pooling học một trọng số α_t cho từng bước, rồi gộp chuỗi thành một vector ngữ cảnh:

$$e_t = w^\top \tanh(W h_t + b), \qquad \alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)}, \qquad v = \sum_{t=1}^{T} \alpha_t h_t$$

1.7. Tăng cường dữ liệu và chuẩn hóa tốc độ

Người ký khác nhau ký nhanh chậm khác nhau. Mọi chuỗi được nội suy tuyến tính về cùng T = 48 bước, và bổ sung đặc trưng vận tốc v_t = x_t − x_{t−1} để mô hình thấy hướng chuyển động. Khi huấn luyện, dữ liệu được tăng cường ngay trên keypoint (không trên ảnh) bằng các phép sau:

- Cắt ngẫu nhiên 80–100% độ dài chuỗi;
- Co dãn ±10%, xoay ±10°, dịch chuyển ±0,1 đơn vị vai;
- Thêm nhiễu Gauss (σ = 0,01);
- Xóa ngẫu nhiên một tay ở 5% số frame, mô phỏng lúc bộ phát hiện bị mất tay.

1.8. Hàm mất mát và tối ưu

Mô hình dùng cross-entropy có trọng số lớp w_y (bù mất cân bằng số mẫu) và label smoothing ε = 0,05 (giảm tự tin thái quá). Tối ưu bằng AdamW, lịch học cosine annealing và early stopping theo validation loss.

$$\mathscr{L} = -\, w_y \sum_{k=1}^{K} \tilde{y}_k \;\log \hat{p}_k\,, \qquad \tilde{y}_k = (1-\varepsilon)\,1[k=y] + \frac{\varepsilon}{K}$$

1.9. Tổng hợp tiếng nói (TTS)

TTS chuyển văn bản thành âm thanh. Các hệ TTS thần kinh hiện nay dùng một mô hình âm học sinh phổ mel từ chuỗi âm vị, rồi một vocoder sinh sóng âm. Với tiếng Việt, hệ thống phải xử lý đúng 6 thanh điệu. Đề tài dùng giọng neural vi-VN của edge-tts, gTTS dự phòng và VieNeu-TTS chạy offline. App Android dùng TTS có sẵn của hệ điều hành.

1.10. Độ đo đánh giá

Với mỗi lớp k, gọi TP, FP, FN là số dương tính đúng, dương tính sai và âm tính sai. Macro-average lấy trung bình đều trên K lớp, nên các lớp ít mẫu cũng có trọng số ngang lớp nhiều mẫu.

$$P_k = \frac{TP_k}{TP_k + FP_k}, \qquad R_k = \frac{TP_k}{TP_k + FN_k}, \qquad F1_k = \frac{2P_kR_k}{P_k + R_k}, \qquad F1_{macro} = \frac{1}{K}\sum_{k=1}^{K} F1_k$$

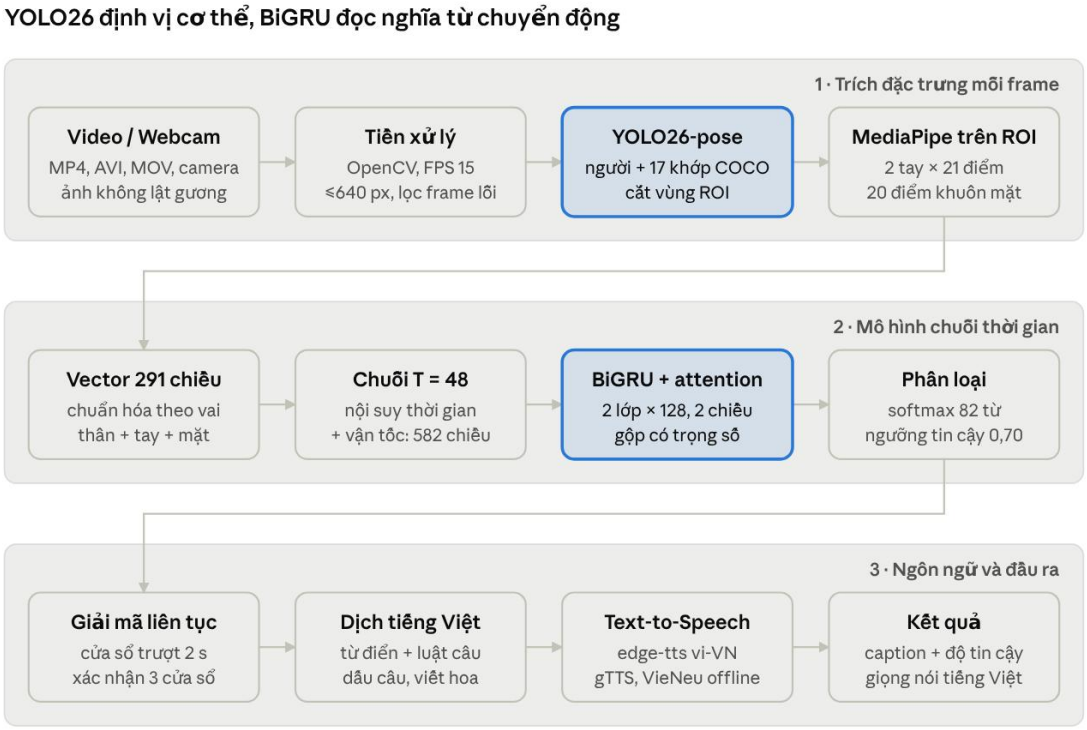
Với câu liên tục, Word Error Rate tính theo số thay thế S, xóa D, chèn I so với N từ tham chiếu (CER tương tự, đếm trên ký tự):

$$\mathrm{WER} = \frac{S + D + I}{N}$$

Chương 2. Đề xuất hệ thống

Hệ thống đề xuất gồm ba giai đoạn. YOLO26 và MediaPipe trích đặc trưng không gian ở từng frame. BiGRU học ý nghĩa từ chuỗi đặc trưng đó. Cuối cùng, tầng ngôn ngữ ghép ký hiệu thành câu tiếng Việt và đọc thành tiếng. Mã nguồn viết bằng Python, chia thành các module độc lập, chạy được trên Windows và Linux, tự dùng GPU CUDA nếu máy có.

2.1. Kiến trúc tổng thể



Hình3.1 Kiến trúc YOLO26

Kiến trúc pipeline · 3 giai đoạn, 12 khối

Hai khối được tô màu là hai mô hình học sâu chính. YOLO26 trả lời câu hỏi "người và tay ở đâu" ở từng frame. BiGRU trả lời câu hỏi "chuyển động này nghĩa là gì" trên cả chuỗi.

2.2. Các module

Bảng 2 Thống kê các module được sử dụng

Module	File	Chức năng

Module	File	Chức năng
Đọc video	src/preprocessing/video_loader.py	Kiểm tra định dạng, chuẩn hóa FPS về 15, resize ≤ 640 px, loại frame đen/lỗi, đọc đường dẫn có dấu trên Windows
Phát hiện	src/detection/yolo_detector.py	YOLO26-pose chọn người ký chính. Có hàm fine-tune YOLO26 trên bộ Hand Keypoints (21 điểm/tay)
Trích đặc trưng	src/detection/keypoint_extractor.py	Cắt ROI từ khung người, chạy MediaPipe Hands/Face, gán tay trái/phải theo cổ tay YOLO26, chuẩn hóa
Chuỗi thời gian	src/recognition/feature_sequence.py	Cache .npy, nội suy T = 48, vận tốc, tăng cường dữ liệu
Mô hình	src/recognition/lstm_model.py	BiGRU/BiLSTM + attention pooling
Phân loại	src/recognition/sign_classifier.py	Nạp checkpoint, kiểm tra cấu hình đặc trưng khớp, trả top-k và độ tin cậy
Dịch	src/translation/vietnamese_translator.py	Gloss → câu tiếng Việt theo labels.yaml và grammar.yaml
Giọng nói	src/speech/vietnamese_tts.py	edge-tts → gTTS → VieNeu, có cache và phát lại
Thời gian thực	src/realtime/webcam_recognition.py	Bộ giải mã liên tục, nhận diện webcam và video
Đánh giá	src/evaluation/evaluate.py	Accuracy, P/R/F1, confusion matrix, độ trễ, FPS, WER/CER
Giao diện	app/main.py	PyQt5: tải video, webcam, phát âm, phát lại, xóa

2.3. Vector đặc trưng mỗi frame

Mỗi frame được biểu diễn bằng một vector 291 chiều. Phần nào không phát hiện được (ví dụ tay ra khỏi khung) được gán 0, kèm cờ có/không có tay.

Bảng 3 Thống kê các vector đặc trưng

Thành phần	Số chiều	Nguồn
Thân trên: 13 khớp × (x, y, conf)	39	YOLO26-pose
Tay trái: 21 × (x, y, z) theo thân + 21 × (x, y) hình dạng cục bộ	105	MediaPipe Hands
Tay phải: như trên	105	MediaPipe Hands
Cờ có tay (trái, phải)	2	—
Khuôn mặt: 20 điểm × (x, y)	40	MediaPipe Face
Tổng	**291**	

Tay được gán trái/phải theo khoảng cách tới cổ tay do YOLO26 dự đoán, thay vì chỉ dựa vào nhãn handedness của MediaPipe (nhãn này giả định ảnh đã lật gương). Video luôn được xử lý ở dạng không lật. Webcam chỉ lật khi hiển thị, để tay trái/phải nhất quán giữa huấn luyện và sử dụng.

2.4. Mô hình nhận diện

Bảng 4 Thống kê các thành phần trong mô hình

Thành phần	Cấu hình
Đầu vào	48 bước × 582 chiều (291 vị trí + 291 vận tốc)
Chiếu đặc trưng	LayerNorm → Linear 256 → LayerNorm → GELU → Dropout 0,3
Chuỗi	BiGRU 2 lớp, 128 đơn vị mỗi chiều (có thể đổi sang BiLSTM trong cấu hình)
Gộp	Attention pooling → vector 256 chiều
Đầu ra	LayerNorm → Dropout → Linear K lớp → softmax
Kích thước	khoảng 0,8 triệu tham số

Thành phần	Cấu hình
Huấn luyện	AdamW (lr 0,001, weight decay 0,0001), batch 32, tối đa 150 epoch, cosine annealing, early stopping sau 25 epoch, label smoothing 0,05, trọng số lớp

2.5. Giải mã liên tục (webcam và video dài)

Mô hình được huấn luyện trên từng từ riêng lẻ. Để nhận một chuỗi ký hiệu liên tiếp, bộ giải mã hoạt động như sau:

1. Giữ buffer đặc trưng của 2 giây gần nhất. Cứ mỗi 3 frame, phân loại lại buffer nếu ít nhất 30% số frame có tay.
2. Chỉ chấp nhận một nhãn khi top-1 giống nhau ở 3 cửa sổ liên tiếp và độ tin cậy ≥ 0,70. Nhãn nền `_nen` không bao giờ được xuất.
3. Sau khi xuất một ký hiệu, xóa buffer. Không xuất lại cùng ký hiệu nếu chưa có khoảng ngắt (cửa sổ khác nhãn hoặc không có tay) và chưa qua 1 giây.
4. Không thấy tay trong 1,2 giây thì kết thúc câu, dịch và đọc cả câu một lần.
5. Với video ngắn chỉ có một ký hiệu: nếu bộ giải mã không bắt được gì, hệ thống phân loại trên toàn bộ video.

2.6. Dịch sang tiếng Việt

Bộ dịch dùng từ điển và luật, kiểm soát được, không tự đoán nghĩa:

- labels.yaml ánh xạ nhãn không dấu sang từ có dấu (benh_vien → "bệnh viện").
- grammar.yaml khai báo cụm gloss → câu, ưu tiên cụm dài nhất. Ví dụ, VSL đặt phủ định cuối câu: [TÔI][ĂN][KHÔNG] → "Tôi không ăn."
- Gộp nhãn lặp, bỏ nhãn nền, thêm dấu câu: "?" khi có từ để hỏi, "!" cho lời chào. Viết hoa đầu câu.
- Ví dụ: [XIN CHÀO][TÔI][KHỎE] → "Xin chào! Tôi khỏe."

2.7. Đọc thành tiếng

Module TTS thử lần lượt edge-tts (giọng neural `vi-VN-HoaiMyNeural`), gTTS, VieNeu-TTS (offline). Engine nào lỗi thì tự chuyển sang engine kế tiếp. Âm thanh được cache theo nội dung, nên câu đã đọc phát lại ngay. Hệ thống đọc cả câu sau khi kết thúc, không đọc từng nhãn riêng lẻ.

Chương 3. Kết quả và thực nghiệm

Trên 568 video test thuộc 82 từ, mô hình BiGRU đạt accuracy 95,42% và F1-macro 0,939. Trong 26 video bị đoán sai, 24 video có độ tin cậy dưới ngưỡng 0,70, nên hệ thống đã từ chối trả lời thay vì đưa ra caption sai.

3.1. Môi trường thực nghiệm

Bảng 5 Các môi trường thực nghiệm

Hạng mục	Cấu hình
Máy	Laptop Windows, chạy CPU (không GPU)
Phần mềm	Python 3.12, PyTorch (bản CPU), Ultralytics, MediaPipe Tasks, OpenCV
Mô hình thị giác	yolo26n-pose.pt (bản nano), MediaPipe Hand Landmarker + Face Landmarker
Tiền xử lý	FPS 15, cạnh dài ≤ 640 px, loại video dưới 8 frame hợp lệ
Mô hình chuỗi	BiGRU 2 lớp × 128 + attention, T = 48 (cấu hình mục 2.4)

3.2. Dữ liệu và cách chia

Dữ liệu ViSignLanguage-Video được nhập bằng tools/import_vsl_datasets.py. Script đổi tên nhãn theo label_mapping.pkl, chuyển sang dạng không dấu, loại clip hỏng, bỏ nhãn có dưới 8 clip, rồi chia phân tầng 70/15/15 theo từng nhãn với seed cố định 42.

Bảng 6 Thành phần trong 2 lần thử nghiệm

Lần thử	Dữ liệu train	Số từ	Test
1 (kết quả báo cáo)	ViSign (70% clip hợp lệ)	82	568 clip ViSign
2 (đánh giá)	ViSign + QIPEDC của các từ trùng: 3.802 mẫu	82	cùng 568 clip ViSign

Lần thử 2 giữ nguyên tập test để so sánh công bằng: video QIPEDC chỉ được đưa vào train.

3.3. Kiểm thử pipeline

Trước khi huấn luyện trên dữ liệu thật, từng thành phần được kiểm thử riêng:

- **Bộ dịch:** 7/7 trường hợp đúng, gồm gộp nhãn lặp, bỏ nhãn nền, câu hỏi, phủ định cuối câu và từ chưa có trong từ điển.
- **Bộ giải mã liên tục:** với kịch bản "ký hiệu A → chuyển → ký hiệu B → nghỉ", bộ giải mã xuất đúng [A, B] và kết thúc một câu. Kiểm thử này đã phát hiện và giúp sửa lỗi cùng một ký hiệu bị xuất 3 lần.
- **Pipeline đầu–cuối:** chạy trên 100 video tổng hợp (ảnh người được xoay/lắc/co dãn). Mục đích chỉ là xác nhận các module kết nối đúng, không dùng để đo độ chính xác nhận diện ký hiệu.
- **Dịch vụ API:** /predict trả đúng JSON với video hợp lệ, và báo lỗi rõ ràng với file hỏng hoặc sai định dạng. Nhờ khởi động mô hình trước, request đầu tiên giảm từ 21,7 s xuống 4,7 s (video 2,5 s, CPU máy chủ thử nghiệm).
- **Giao diện PyQt5:** nạp mô hình, xử lý video tải lên, hiển thị khung xương, độ tin cậy, top-3 và xóa kết quả.

3.4. Kết quả nhận diện trên tập test

Bảng 7 Kết quả trên tập Test

Độ đo	Macro	Weighted
Precision	95,00%	96,28%
Recall	94,47%	95,42%
F1-score	93,94%	95,44%

Accuracy đạt 95,42% (542/568 video), cross-entropy loss trên test 0,266. F1-macro thấp hơn F1-weighted khoảng 1,5 điểm, cho thấy một số từ ít mẫu được nhận diện kém hơn trung bình. Danh sách cụ thể các từ này nằm trong runs/evaluation/classification_report.txt.

3.5. Hiệu quả của ngưỡng tin cậy

Bảng 8 Phân tích ngưỡng tin cậy

Nhóm video test	Số video	Đúng	Sai
Độ tin cậy ≥ 0,70 (có caption)	517 (91,0%)	515	2

Nhóm video test	Số video	Đúng	Sai
Độ tin cậy < 0,70 (từ chối)	51 (9,0%)	27	24
Tổng	**568**	**542**	**26**

Khi đã xuất caption, hệ thống đúng 99,6%. Ngược lại, hơn một nửa số video bị từ chối thực ra đã được đoán đúng. Hạ ngưỡng sẽ tăng số video có caption nhưng cũng tăng số caption sai. Ngưỡng 0,70 phù hợp cho một công cụ giao tiếp, nơi một câu dịch sai gây hại nhiều hơn một lần yêu cầu ký lại.

3.6. Tốc độ xử lý

Trên CPU, trích đặc trưng (YOLO26 + MediaPipe) mất 102,6 ms mỗi frame. Phân loại bằng BiGRU mất 13,5 ms mỗi cửa sổ và chạy mỗi 3 frame. Toàn bộ pipeline đạt khoảng 9,3 FPS. Bước trích đặc trưng chiếm khoảng 96% thời gian, nên muốn tăng tốc phải xử lý ở bước này: dùng GPU, chạy YOLO26 cách 2 frame (detect_interval: 2), hoặc tắt đặc trưng khuôn mặt.

3.7. Biểu đồ kết quả

Các hình sau được sinh tự động trong file runs/evaluation/:

- loss_curve.png: training loss và validation loss theo epoch.
- accuracy_curve.png: training accuracy và validation accuracy theo epoch.
- confusion_matrix.png: ma trận nhầm lẫn 82 × 82 trên tập test.
- latency.png: thời gian trích đặc trưng và phân loại.

3.8. Thảo luận và hạn chế

- **Kết quả có thể cao hơn thực tế.** ViSign không ghi mã người ký, nên dữ liệu phải chia ngẫu nhiên, và cùng một người có thể xuất hiện cả ở train lẫn test. Độ chính xác với người lạ trước camera nhiều khả năng thấp hơn 95%.
- **Độ phân giải thấp:** video ViSign chỉ 224 × 224 px, nên MediaPipe khó định vị chính xác ngón tay.
- **Thiếu nhãn nền _nen:** khi người dùng không ký gì trước webcam, mô hình vẫn cố đoán một từ. Ngưỡng tin cậy chỉ giảm được phần nào.
- **Từ vựng hẹp:** 82 từ đủ để demo theo chủ đề, chưa đủ cho giao tiếp hằng ngày.
- **Dịch theo luật:** chỉ xử lý thứ tự từ VSL ở các mẫu đã khai báo. Các câu ngoài mẫu được ghép theo kiểu nối từ.
- **Chưa đo WER/CER:** chưa có video câu liên tục kèm câu tham chiếu. Công cụ đo đã có sẵn (evaluate.py --sentences).

3.9. Thực nghiệm đang thực hiện

- **Gộp QIPEDC (lần thử 2):** đã huấn luyện với 3.802 mẫu train, đang so F1-macro với mốc 0,939 của lần thử 1.
- **VieSign-SI447:** 447 từ, 50 người ký, train/val/test không trùng người (25/13/12 người). Đây sẽ là phép đánh giá trên người ký chưa từng thấy, phản ánh đúng điều kiện sử dụng thật.
- **Kiểm tra với người dùng thật:** quay video của người chưa có trong dữ liệu, so kết quả giữa các mô hình bằng predict.py.

Kết luận

Đề tài đã xây dựng hệ thống phiên dịch ngôn ngữ ký hiệu, đi từ video ngôn ngữ ký hiệu tới câu tiếng Việt và giọng nói. Hệ thống đạt accuracy 95,42% trên 82 từ và có ba cách sử dụng: dòng lệnh, giao diện desktop.

1. Kết quả đạt được

- Kiến trúc kết hợp **YOLO26-pose** (phát hiện người, khung xương, vùng ROI) với **MediaPipe** (21 điểm/tay, biểu cảm) và **BiGRU + attention** (ý nghĩa theo thời gian). Kiến trúc này đáp ứng đúng yêu cầu không để YOLO tự hiểu chuỗi ký hiệu.
- Vector đặc trưng 291 chiều, chuẩn hóa theo vai, không phụ thuộc vị trí đứng, khoảng cách hay tốc độ ký.
- Kết quả trên 568 video test: accuracy 95,42%, F1-macro 0,939. Với ngưỡng 0,70, caption đúng 99,6% trên 91% số video được trả lời.
- Nhận diện thời gian thực khoảng 9 FPS trên CPU, có cơ chế chống lặp và tự kết thúc câu.
- Dịch tiếng Việt theo từ điển và luật, kiểm soát được. TTS tiếng Việt có nhiều engine dự phòng, cache và phát lại.
- Bộ công cụ dữ liệu: quay dataset bằng webcam, chia theo người ký, nhập và gộp nhiều bộ dữ liệu, kiểm tra cấu trúc, kiểm thử tự động.

2. Hướng phát triển

- Mở rộng dữ liệu và từ vựng. Huấn luyện trên VieSign-SI447 (447 từ, 50 người ký). Xin quyền truy cập VSL400 (74.259 clip, 400 từ, 28 người ký). Bổ sung nhãn nền _nen và video tự quay của người dùng thật.
- Đánh giá theo người ký. Báo cáo kết quả trên người ký chưa từng thấy làm thước đo chính.
- Dùng YOLO26 cho cả bàn tay. Fine-tune YOLO26-pose trên bộ Hand Keypoints (lệnh train.py yolo-hands đã có), thay MediaPipe Hands.
- Mô hình mạnh hơn. Thử Transformer hoặc mạng đồ thị trên khung xương (ST-GCN), và kết hợp nhiều góc quay khi có dữ liệu multi-view.
- Nhận diện và dịch câu liên tục. Huấn luyện với CTC trên video câu. Thay luật dịch bằng mô hình ngôn ngữ khi có corpus song song gloss–câu. Đo WER/CER.
- Xây dựng ứng dụng chạy trực tiếp trên điện thoại. Xuất YOLO26 và BiGRU sang TFLite, viết lại phần chuẩn hóa đặc trưng trên thiết bị, để dịch thời gian thực qua camera mà không cần mạng.

- Chiều ngược lại. Chuyển văn bản hoặc giọng nói thành ký hiệu (avatar 3D) để giao tiếp hai chiều.

Tài liệu tham khảo

1. Ultralytics. *YOLO26 — Unified Real-Time End-to-End Vision Models* (tài liệu mô hình). docs.ultralytics.com/models/yolo26
2. Ultralytics. *Pose Estimation* (tác vụ YOLO26-pose, 17 keypoint COCO). docs.ultralytics.com/tasks/pose
3. *Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8, and YOLOv5 Object Detectors*. arXiv:2510.09653. arxiv.org/abs/2510.09653
4. Lugaresi, C. và cộng sự (2019). *MediaPipe: A Framework for Building Perception Pipelines*. arXiv:1906.08172.
5. Zhang, F. và cộng sự (2020). *MediaPipe Hands: On-device Real-time Hand Tracking*. arXiv:2006.10214.
6. Hochreiter, S., Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.
7. Cho, K. và cộng sự (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP 2014, 1724–1734.
8. Bahdanau, D., Cho, K., Bengio, Y. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR 2015.
9. Loshchilov, I., Hutter, F. (2019). *Decoupled Weight Decay Regularization*. ICLR 2019.
10. Loshchilov, I., Hutter, F. (2017). *SGDR: Stochastic Gradient Descent with Warm Restarts*. ICLR 2017.
11. Szegedy, C. và cộng sự (2016). *Rethinking the Inception Architecture for Computer Vision*. CVPR 2016 (label smoothing).
12. Li, D., Rodriguez, C., Yu, X., Li, H. (2020). *Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison*. WACV 2020, 1459–1469.
13. Camgoz, N. C. và cộng sự (2018). *Neural Sign Language Translation*. CVPR 2018, 7784–7793.
14. Dinh, N. S. và cộng sự (2025). *Sign Language Recognition: A Large-Scale Multi-View Dataset and Comprehensive Evaluation*. WACV 2025, 7876–7886. openaccess.thecvf.com
15. Dang, K., Bessmertny, I. A. (2024). *ViSL One-shot: generating Vietnamese sign language data set*. Scientific and Technical Journal of Information Technologies, Mechanics and Optics, 24(2), 241–248. openbooks.itmo.ru
16. Trương Duy Việt và cộng sự. *MoViNet-A2 cho bài toán nhận diện ký hiệu Tiếng Việt*. Tạp chí Khoa học ĐH Mở TP.HCM – Kỹ thuật và Công nghệ, 20(2), 3–14. vjol.info.vn

17. *ViSignLanguage-Video* (bộ dữ liệu, CC BY 4.0). huggingface.co/datasets/star092304/ViSignLanguage-Video
18. Dự án QIPEDC – Bộ Giáo dục và Đào tạo. *Từ điển Ngôn ngữ Ký hiệu Việt Nam*. qipedc.moet.gov.vn; bản thu thập trên Kaggle: VSL-Vietnamese Sign Languages
19. *VieSign-10K: A Vietnamese Isolated Sign Language Dataset with Original and Signer-Independent Benchmark Splits*. Zenodo, CC BY 4.0. zenodo.org/records/23610105
20. VieNeu-TTS — Vietnamese Text-to-Speech on-device. pypi.org/project/vieneu; edge-tts. pypi.org/project/edge-tts